

Complete Git & GitHub Learning Guide — From Scratch to Real-World Scenarios

I want to learn **Git and GitHub** from absolute beginner level to advanced, real-world usage.

Create a **complete, structured learning guide** that teaches me Git from scratch. Assume I have **never used Git before**.

The goal is not just to memorize commands. I want to understand **what Git is doing, why each command is used, when to use it, and what to do when something goes wrong**.

1. Git Setup — Start From Zero

Start with installation and configuration.

Cover:

- What is Git?
- Git vs GitHub
- Why developers use Git
- Version control explained simply
- Installing Git on Windows
- Checking whether Git is installed
- Configuring username
- Configuring email
- Configuring default branch
- Git Bash basics
- Important terminal commands:
 - `cd`
 - `ls`
 - `pwd`
 - `mkdir`
 - `touch`
 - `clear`
 - `rm`
- Checking Git configuration
- Understanding `.gitconfig`

Show the exact commands I should run.

2. Git Fundamentals

Explain the Git workflow visually and practically:

Working Directory → Staging Area → Local Repository → Remote Repository

Explain:

- Working directory
- Staging area
- Local repository
- Remote repository
- Commit
- Branch
- HEAD
- Repository
- `.git` folder
- SHA/hash
- Snapshot concept

For every concept, give a simple real-world analogy.

3. First Git Repository

Teach me how to create my first repository from scratch.

Example:

```
mkdir my-project
cd my-project
git init
```

Then demonstrate:

```
git status
git add
git commit
git log
```

Explain exactly what happens after every command.

Show a complete first-project example.

4. Git Basic Commands

Teach each command with:

1. What it does
2. Why we use it
3. Syntax
4. Simple example
5. Expected result
6. Common mistakes
7. When NOT to use it

Cover at minimum:

```
git init
git status
git add
git commit
git log
git diff
git show
git rm
git mv
git restore
git clean
```

Also explain:

```
git add .
git add -A
git add <file>
git commit -m ""
git commit -am ""
```

Clearly explain the differences between them.

5. Git Log & History

Teach me how to inspect project history.

Cover:

```
git log
git log --oneline
git log --graph
git log --all
git log --oneline --graph --decorate --all
git show
git diff
```

Explain how to find:

- Previous commits
- What changed in a commit
- Which files changed
- Commit IDs
- Branch history

6. Git Branches — Complete Guide

Start from the basics:

- What is a branch?
- Why branches are needed
- `main`
- Creating a branch
- Switching branches
- Creating and switching in one command
- Listing branches
- Renaming branches
- Deleting branches

Commands:

```
git branch
git branch feature-login
git switch feature-login
git switch -c feature-login
git checkout
git branch -m
git branch -d
```

Explain `git switch` vs `git checkout`.

Give realistic examples such as:

```
main
├─ feature-login
├─ feature-payment
└─ bugfix-navbar
```

7. Merging Branches

Teach:

```
git merge
```

Explain:

- Fast-forward merge
- Three-way merge
- Merge commit
- When merge conflicts occur
- How Git determines conflicts

Create a complete example where:

```
main

feature
```

is merged back into `main`.

8. Merge Conflicts — VERY IMPORTANT

Teach conflicts from scratch.

Create an example where two branches modify the same line.

Show the conflict markers:

```
<<<<<<< HEAD
your code
=====
```

```
other branch code
>>>>>> feature
```

Explain:

- What each marker means
- How to manually resolve the conflict
- How to stage the resolved file
- How to complete the merge
- How to abort a merge

Commands:

```
git status
git add .
git commit
git merge --abort
```

Also explain what to do if the conflict occurs in:

- JavaScript
- React
- Java
- JSON
- package-lock.json

9. GitHub Setup

Teach GitHub from scratch.

Explain:

- What GitHub is
- Git vs GitHub
- Creating a GitHub account
- Creating a repository
- Public vs private repository
- Repository README
- `.gitignore`
- LICENSE

Show how to connect a local repository to GitHub.

Commands:

```
git remote
git remote -v
git remote add origin <url>
git push
git push -u origin main
```

Explain exactly what `origin` means.

10. Push, Pull & Fetch

Teach the difference between:

```
git push
git pull
git fetch
```

Explain:

fetch = download information

pull = fetch + merge/rebase

push = upload local commits

Give realistic examples.

11. Clone an Existing Project

Teach:

```
git clone <url>
```

Then explain the complete workflow after cloning:

```
git clone
cd project
git status
```

```
git branch
git pull
```

Show how a developer starts working on an existing company project.

12. Remote Repositories

Teach:

```
git remote -v
git remote add
git remote remove
git remote rename
git remote set-url
git fetch
```

Explain local branches vs remote-tracking branches:

```
main
origin/main
feature/login
origin/feature/login
```

13. GitHub Authentication

Explain modern GitHub authentication.

Cover:

- HTTPS
- SSH
- SSH keys
- Personal Access Tokens
- Why GitHub no longer accepts normal passwords for Git operations
- How to configure SSH on Windows
- How to test SSH connection

Show commands such as:


```
ssh-keygen  
ssh-add  
ssh -T git@github.com
```

Explain when HTTPS vs SSH should be used.

14. .gitignore

Teach `.gitignore` properly.

Show examples for:

- Node.js
- React
- Java
- IntelliJ
- VS Code
- Environment variables
- Build folders
- Dependencies

Example:

```
node_modules/  
.env  
dist/  
build/  
*.log
```

Explain what happens if a file was already committed before adding it to `.gitignore`.

Show how to remove it from Git tracking without deleting it locally.

15. Git Stash

Teach:

```
git stash  
git stash list  
git stash pop
```

```
git stash apply
git stash drop
git stash clear
```

Create realistic situations:

Scenario 1

You are working on a feature but suddenly need to fix production code.

Scenario 2

You have uncommitted changes and need to switch branches.

Scenario 3

You want to stash only selected files.

Explain when stash should and should not be used.

16. Undoing Changes — COMPLETE GUIDE

This section should be extremely detailed.

Explain the difference between:

```
git restore
git reset
git revert
```

Cover:

Undo unstaged changes

```
git restore file
```

Unstage a file

```
git restore --staged file
```

Undo the last commit but keep changes

```
git reset --soft HEAD~1
```

Undo commit and keep changes unstaged

```
git reset --mixed HEAD~1
```

Completely remove commit and changes

```
git reset --hard HEAD~1
```

Safely undo a pushed commit

```
git revert <commit>
```

Explain **when each should be used**.

Especially explain why `git reset --hard` is dangerous.

17. Git Reset — Deep Understanding

Teach:

```
git reset --soft  
git reset --mixed  
git reset --hard
```

Use diagrams to show:

```
HEAD  
↓  
Commit  
↓  
Staging Area  
↓  
Working Directory
```

Show exactly what each reset option changes.

18. Git Revert

Explain why revert is preferred for shared/public history.

Example:

```
A → B → C
```

If C is bad:

```
A → B → C → D
```

where D reverses C.

Explain why we don't simply reset a shared branch.

19. Git Rebase

Teach rebase from scratch.

Explain:

- What is rebase?
- Merge vs rebase
- Why teams use rebase
- Interactive rebase

Commands:

```
git rebase main  
git rebase -i HEAD~3
```

Explain:

```
pick  
reword
```

```
edit  
squash  
fixup  
drop
```

Show a realistic example of cleaning up messy commits.

20. Rebase Conflicts

Create a realistic conflict during:

```
git rebase
```

Teach:

```
git status
```

Resolve the conflict and then:

```
git add .  
git rebase --continue
```

Also teach:

```
git rebase --abort
```

Explain the difference between merge conflicts and rebase conflicts.

21. Cherry-Pick

Teach:

```
git cherry-pick <commit>
```

Explain when it is useful.

Give realistic examples such as:

- A bug fix exists on another branch
- You only need one commit
- Backporting a fix to another branch

Also explain cherry-pick conflicts.

22. Git Tags

Teach:

```
git tag
git tag v1.0.0
git tag -a v1.0.0 -m "Release 1.0.0"
git push origin v1.0.0
git push origin --tags
```

Explain semantic versioning:

```
MAJOR.MINOR.PATCH
```

23. GitHub Pull Requests

Teach the complete professional workflow:

```
clone
↓
create branch
↓
make changes
↓
commit
↓
push branch
↓
create Pull Request
↓
code review
↓
```

```
changes requested
↓
update branch
↓
merge PR
```

Explain:

- Pull Request
- Code review
- Review comments
- Approvals
- Merge
- Squash and merge
- Rebase and merge
- Close PR
- Delete branch

24. Real-World Team Workflow

Teach a realistic company workflow.

Example:

```
main
develop
feature/*
bugfix/*
hotfix/*
```

Explain different branching strategies:

- Feature branching
- Git Flow
- Trunk-based development

Show what a developer typically does every morning:

```
git switch main
git pull
git switch -c feature/login
```

Then throughout the day:

```
git status
git add .
git commit
git push
```

25. Common Real-World Scenarios

This is one of the most important sections.

For EVERY scenario:

1. Explain the problem
2. Explain why it happened
3. Show the exact commands
4. Explain what each command does
5. Show the safest solution
6. Mention dangerous alternatives

Include at least these scenarios:

Scenario 1

I accidentally committed the wrong file.

Scenario 2

I committed `.env`.

Scenario 3

I made changes on the wrong branch.

Scenario 4

I forgot to create a branch before coding.

Scenario 5

I need to undo my last commit.

Scenario 6

I need to undo a commit that was already pushed.

Scenario 7

I accidentally deleted a file.

Scenario 8

I deleted a branch accidentally.

Scenario 9

I have uncommitted changes but need to switch branches.

Scenario 10

I have a merge conflict.

Scenario 11

I have a rebase conflict.

Scenario 12

Someone pushed changes to the branch while I was working.

Scenario 13

My local branch is behind remote.

Scenario 14

My local branch and remote branch have diverged.

Scenario 15

I pushed to the wrong branch.

Scenario 16

I committed several small commits and want to combine them.

Scenario 17

I need only one commit from another branch.

Scenario 18

I accidentally ran `git reset --hard`.

Scenario 19

I need to find who changed a particular line.

Scenario 20

I need to find which commit introduced a bug.

Scenario 21

I cloned a repository and cannot push.

Scenario 22

Git says there are conflicts in `package-lock.json`.

Scenario 23

I changed files but `git status` does not show them.

Scenario 24

Git is tracking a file that is now in `.gitignore`.

Scenario 25

I need to rename a branch.

Scenario 26

I need to delete a remote branch.

Scenario 27

I need to change the remote repository URL.

Scenario 28

I accidentally committed directly to `main`.

Scenario 29

I need to temporarily save work without committing.

Scenario 30

I need to recover a deleted/lost commit.

26. Git Reflog

Teach:

```
git reflog
```

Explain why reflog is extremely useful for recovery.

Show how to recover from:

- Accidental reset
- Deleted branch
- Lost commit
- Bad rebase

Explain that reflog is one of the most important Git recovery tools.

27. Git Bisect

Teach:

```
git bisect start  
git bisect bad  
git bisect good
```

Show how to find the commit that introduced a bug.

Use a simple example.

28. Git Blame

Teach:

```
git blame filename
```

Explain:

- What it shows
 - When it is useful
 - Why it should not be used to "blame" developers personally
-

29. Git Diff — Advanced

Teach:

```
git diff
git diff --staged
git diff HEAD
git diff branch1 branch2
```

Explain exactly what each comparison means.

30. Commit Best Practices

Teach:

- Good commit messages
- Atomic commits
- Small commits
- What NOT to commit
- Commit frequency
- Conventional commits

Examples:

```
feat: add login form
fix: resolve navbar overflow
docs: update README
refactor: simplify authentication logic
```

31. GitHub Best Practices

Explain:

- README
 - `.gitignore`
 - Branch protection
 - Pull Requests
 - Code reviews
 - Protected `main`
 - Secrets
 - GitHub Actions introduction
 - Issues
 - Releases
-

32. GitHub Actions — Beginner Introduction

Give me a basic introduction to CI/CD.

Explain:

- What CI/CD means
- What GitHub Actions is
- Workflow
- Job
- Step
- Trigger

Show a simple workflow:

```
name: CI

on:
  push:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
```

```
- run: npm install
- run: npm test
```

Explain every line.

33. Git GUI Tools

Briefly explain:

- Git Bash
- VS Code Git integration
- GitHub Desktop
- Git GUI tools

Explain what beginners should understand before relying on GUI tools.

34. Git Commands Cheat Sheet

At the end, create a concise cheat sheet organized into:

Setup

Create Repository

Basic Workflow

Branches

Remote

Merge

Rebase

Stash

Undo

Recovery

GitHub

Debugging

For every command, provide a one-line explanation.

35. Git Decision Guide

Create a section:

"Which Git Command Should I Use?"

Use practical questions.

For example:

I want to undo local changes → ?

```
git restore
```

I want to undo a public commit → ?

```
git revert
```

I want to temporarily save work → ?

```
git stash
```

I want one commit from another branch → ?

```
git cherry-pick
```

I want to find a lost commit → ?

```
git reflog
```

I want to combine commits → ?

```
git rebase -i
```

This should become a quick decision-making reference.

36. Practice Projects

Create practical exercises after teaching the concepts.

Project 1 — Personal Git Repository

Create a simple project and practice:

- init
- add
- commit
- branch
- merge
- push

Project 2 — Team Simulation

Simulate two developers working on the same project.

Practice:

- branches
- push
- pull
- conflicts
- Pull Requests

Project 3 — Git Disaster Recovery

Intentionally create mistakes:

- wrong commit
- wrong branch
- merge conflict
- reset
- deleted branch

Then recover everything using Git.

37. Interview Preparation

At the end, include Git interview questions.

Divide them into:

Beginner

Intermediate

Advanced

Scenario-Based

For each question provide:

- Simple interview answer
- Technical explanation
- Example if necessary

Include questions such as:

- What is Git?
- Git vs GitHub?
- What is a commit?
- What is staging?
- What is a branch?
- Merge vs rebase?
- Pull vs fetch?
- Reset vs revert?
- What is cherry-pick?
- What is stash?
- What is reflog?
- How do you resolve merge conflicts?

- What happens during `git pull`?
 - What is HEAD?
 - What is a detached HEAD?
 - What is `.gitignore`?
 - What is a Pull Request?
 - How do you recover a deleted commit?
-

38. Learning Format

Follow these rules throughout the guide:

Explain Like I'm a Beginner

Do not assume prior Git knowledge.

Every command must have an example

Use real commands rather than only definitions.

Explain the output

Whenever useful, show what Git is expected to display.

Use diagrams

Use simple ASCII diagrams such as:

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Local Repository
  ↓
git push
  ↓
GitHub
```

Use realistic developer scenarios

Do not teach Git only through artificial examples.

Use situations developers actually face.

Warn about dangerous commands

Clearly mark commands such as:

```
git reset --hard
git push --force
git clean -fd
```

Explain when they are safe and when they can cause data loss.

Compare similar commands

Whenever commands are commonly confused, create a comparison.

Examples:

Command	Purpose
<code>git fetch</code>	Download remote information
<code>git pull</code>	Fetch + integrate changes
<code>git push</code>	Upload commits

Do this for:

- reset vs revert
- merge vs rebase
- pull vs fetch
- restore vs reset
- stash apply vs stash pop
- switch vs checkout

39. Final Learning Path

Structure the entire guide in this order:

```
LEVEL 1 – Git Basics
↓
LEVEL 2 – Local Repository
↓
```

```
LEVEL 3 – Branches
↓
LEVEL 4 – GitHub
↓
LEVEL 5 – Collaboration
↓
LEVEL 6 – Conflicts
↓
LEVEL 7 – Undo & Recovery
↓
LEVEL 8 – Rebase & Advanced Git
↓
LEVEL 9 – Real-World Scenarios
↓
LEVEL 10 – Professional Workflow
↓
LEVEL 11 – Practice Projects
↓
LEVEL 12 – Interview Preparation
↓
FINAL CHEAT SHEET
```

Make the guide **practical enough that after completing it I can confidently use Git in a real software-development team**, create repositories, work with branches, collaborate through GitHub, resolve conflicts, undo mistakes, recover lost work, and understand what Git is doing internally.

Keep explanations beginner-friendly but technically accurate. Do not skip difficult scenarios just because they are advanced.